Basics

Calepin turns ordinary Typst documents into computational notebooks. You write prose, equations, figures, tables, and page layout in Typst, then place executable code directly in the same `.typ` file. When you run `calepin compile notebook.typ`, *Calepin* scans the document for chunks, runs them, stores their results, and asks Typst to render the final document with those results inserted in place.

Standard Typst

A *Calepin* notebook is still a standard Typst document. Headings, paragraphs, lists, math, links, functions, variables, and layout rules are normal Typst. The notebook behavior comes from a small set of imported helper functions and from fenced code blocks that *Calepin* sees before Typst renders the document.

That means a notebook can begin like any other Typst file:

```
#import "/.calepin/calepin.typ" as calepin
#show: calepin.document

#set document(title: [My analysis])

= Introduction

This is regular Typst prose. The expression #math.equation(alt: "pi r squared", block: false, $pi r^2$)
is regular Typst math.
```

During compilation, *Calepin* creates a regenerable `.calepin` directory beside the document. The generic import above loads the generated Typst runtime and stored results from that directory.

Use this generated import rather than the `calepin` placeholder package on Typst Universe.

Code chunks

The simplest executable chunk is an ordinary fenced code block named after its language. *Calepin* runs the block and places the captured output below the source:

````
```python
print(40 + 2)
```
````

```python
print(40 + 2)
```

```
42
```

Languages run in persistent sessions, so variables defined in one chunk are available in later chunks:

````
```python
x = 41
```

```python
print(x + 1)
```
````

```python
x = 41
```

```python
print(x + 1)
```

```
42
```

Set document-wide defaults with `#calepin.setup(...)`. For example, this page uses `results: "verbatim"` so console output is shown as plain text. Other pages use `results: "render"` when plots, rich values, or Typst output should be rendered more fully.

Inline results

Inline code is for computed values that belong inside a sentence. The common pattern is to create a short alias once, then call it where the result should appear:

```
#let py = calepin.inline.with("python")

The answer is #py[`print(40 + 2)`].
```

The answer is 42.

The simple computational notebook on the homepage combines these pieces in one complete, copyable file.

Plain Typst preview

Run Calepin once to generate the runtime and results:

```
calepin compile paper.typ
```

Afterward, ordinary Typst tooling can render the original notebook without extra `--input` arguments:

```
typst compile paper.typ
```

The generic import uses the artifacts from the most recently compiled or watched notebook when Calepin itself is not driving Typst. The document show rule replaces executable raw fences with their stored results, so ordinary Typst edits refresh normally in the preview.

Typst tooling does not evaluate notebook code. After changing Python, R, Julia, shell, Jupyter, or diagram code, run Calepin again, or keep `calepin watch paper.typ --eval-only` running to refresh the results when the computational fingerprint changes.

Compiling another notebook changes the generic import's active fallback. When several notebook previews must remain independent, use the generated notebook-specific facade:

```
#import "/.calepin/paper/calepin.typ" as calepin
#show: calepin.document
```

For `chapters/intro.typ`, the corresponding path is `/.calepin/chapters/intro/calepin.typ`. A notebook-specific facade always uses that notebook's artifacts and ignores which notebook is active. Both import forms continue to work when Calepin drives Typst; Calepin's internal inputs take precedence for the generic form.

Avoid wildcard imports such as `#import "/.calepin/calepin.typ": *`: the exported `document` adapter would shadow Typst's built-in `document` element and break rules such as `#set document(...)`.

See Editor integration for editor-specific setup. Deleting `.calepin` removes the generated runtime and results; run `calepin compile` again before restarting the preview.